Chapter 9

Clustering: K-means, hierarchical clustering, and Gaussian mixtures

1. Unsupervised learning *vs.* Supervised learning

Supervised learning. Up until now, we have concerned ourselves with methods in *supervised learning*.

In supervised tasks, we have N samples which we view as input-output (predictor-observable) pairs, i.e., we define a vector of input variables $\mathbf{x}^{(i)}$ (discrete or continuous) and an output variable $y^{(i)}$:

$$\{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^{N}, \qquad \text{where} \quad \mathbf{x}^{(i)} \in \mathbb{R}^p \quad \text{or} \quad \mathbf{x}^{(i)} \in A_1 \times A_2 \cdots \times A_p \tag{9.1}$$

$$\text{and} \begin{cases} y^{(i)} \in \mathbb{R} & \text{(regression)} \\ \qquad \text{or} & \\ y^{(i)} \in \{c_1, \ldots, c_Q\} & \text{(classification)} \end{cases}$$

where $\{A_m\}_{m=1}^{p}$ are p sets of discrete values that p categorical predictors can take. In all the supervised methods, we take the set $\mathcal{S} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^{N^{\text{training}}}$ to be used for training and cross-validation, i.e., to learn the parameters (and hyper-parameters) $\boldsymbol{\theta}$ of the model $f_{\boldsymbol{\theta}}(\mathbf{x})$.

Once the training and cross-validation is finished, the resulting model can be used to make predictions on unseen samples, i.e., given a new sample $\mathbf{x}^{\text{in}}$ we use the model to predict the output $\widehat{y}$ (either its class assignment or its value):

$$\text{Given } \mathbf{x}^{\text{in}}, \quad \text{predict } \widehat{y} = \begin{cases} f_{\boldsymbol{\theta}}(\mathbf{x}^{\text{in}}) \in \mathbb{R} & \text{(regression)} \\ \qquad \text{or} & \\ f_{\boldsymbol{\theta}}(\mathbf{x}^{\text{in}}) \in \{c_1, \ldots, c_Q\} & \text{(classification)} \end{cases}$$

In other words, in supervised learning we have samples that provide a *ground truth*, i.e., we take the observations of the output variable $y^{(i)}$ to be true, and our target is to learn from those *true* examples so that we can generalise the task to *unseen* samples.

Unsupervised learning. In the following chapters, we turn our attention towards *unsupervised learning*. In this type of learning tasks, there is a dataset of interest, but

we do **not** view it as a collection of input-output pairs where the target is to predict the output from the input. We now we consider all the variables equally, i.e., we do not define a separation between descriptors (or predictors) and outputs to predict. Our dataset is now viewed only as a collection of (continuous or discrete) samples:

$$\{\mathbf{x}^{(i)}\}_{i=1}^{N}, \qquad \mathbf{x}^{(i)} \in \mathbb{R}^p \quad \text{or} \quad \mathbf{x}^{(i)} \in\in A_1 \times A_2 \cdots \times A_p. \tag{9.2}$$

As a consequence, we do not treat our data as a training set that provides a ground truth for the prediction of some output. Hence, there is no training against known outputs, as we have seen in supervised learning. Likewise, we will not have measures of success computed as the quality of our predictions, since there is no ground truth against which we can assess the goodness of the models.

The main idea of unsupervised learning is to learn the intrinsic structure of the data, as it emerges from the data itself. The target is to discover informative and concise representations of the data that can help our understanding of the observations, and which can drive data-informed discovery from our observations. This implies that the objectives and quality measures arising in unsupervised learning are more open-ended. Yet, they can still be mathematically and computationally formulated, usually as optimisation tasks.

In the next chapters, we will concentrate on two of the most important types of tasks in unsupervised learning:

- **Clustering:** find groups of data points such that the samples within each group are more similar to each other as compared to samples outside of the group. A good clustering will allow us to describe our dataset in terms of the obtained groups (or 'clusters') in a concise manner. The clusters found can also drive interpretation and discovery through the investigation of the meaning of such groups of samples present in the dataset.
- **Dimensionality reduction:** find a 'good' approximate representation of the original dataset (9.2) in terms of lower dimensional variables $\mathbf{a}^{(i)} \in \mathbb{R}^m$ where $m \ll p$. This is important when we have high-dimensional data (i.e., p is very large so that each sample is described by many variables) and we want to find a more concise description in terms of fewer variables obtained (in many cases non-trivially) from the original variables. Dimensionality reduction can also improve interpretability of data and models by eliminating variables that are not very descriptive, instead focussing on those variables that capture the most relevant properties of the dataset.

In this chapter, we first introduce and define the pre-requisites common to all clustering algorithms, before showcasing these approaches with three of the most established clustering methods.

2. Similarity measures for clustering

Usually, the first step towards clustering data is to decide on the notion and definition of *similarity*, $S(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$, or *dissimilarity*, $D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$, between data points. This notion of dissimilarity is oftentimes closely related to a *distance*, but many (dis)similarities are not true metrics (i.e., they do not fulfil all the properties of a metric).

Continuous data: Given a continuous dataset $\mathbf{x} \in \mathbb{R}^p$, typical examples of *dissimilarity* include:

- the squared Euclidean distance (i.e., the L_2-norm):
$$D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2$$
- or the Manhattan distance (i.e., the L_1-norm):
$$D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = |\mathbf{x}^{(i)} - \mathbf{x}^{(j)}|$$

whereas classic examples of *similarity* include:

- the *cosine similarity*:
$$S(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \frac{\mathbf{x}^{(i)} \cdot \mathbf{x}^{(j)}}{\|\mathbf{x}^{(i)}\|\|\mathbf{x}^{(j)}\|}$$
- or measures of a statistical nature (e.g., correlations and covariances), where two variables being correlated implies similarity. For example:
$$S(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \frac{\text{cov}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})}{\sqrt{\text{cov}(\mathbf{x}^{(i)}, \mathbf{x}^{(i)}) \cdot \text{cov}(\mathbf{x}^{(j)}, \mathbf{x}^{(j)})}}$$

Categorical or discrete data: If the variables in the dataset are categorical, $\mathbf{x}^{(i)} \in A_1 \times A_2 \cdots \times A_p$, then we can utilise a distance matrix based on pre-defined 'costs' of a mismatch between categorical values. A particular example of this is the *Hamming distance*, that we had introduced in Chapter 2:

$$D_H(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sum_{m=1}^{p} I(x_m^{(i)} \neq x_m^{(j)})$$

with the indicator function:

$$I(b) = \begin{cases} 1, & \text{if } b \text{ is true} \\ 0, & \text{if } b \text{ is false} \end{cases}$$

The Hamming distance $D_H(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$ assigns a cost = 1 to each mismatch and simply sums them. In other words, it counts the number of positions at which two categorical vectors of the same length differ.

3. Definition of the clustering problem

With a chosen definition of what it means to have 'similar' data, we consider the general mathematical statement of the clustering problem:

> **Key Message:** Given N samples $\{\mathbf{x}^{(i)}\}_{i=1}^{N}$ and a 'distance' $D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$, quantifying dissimilarity, the objective is to find a *partition* of the N samples into K clusters such that the dissimilarity is smaller within clusters than across clusters.

Note again the lack of predictor/outcome pairs, i.e., there are no pre-defined class labels to be learnt, i.e., the partition of the data emerges naturally from the dataset itself.

We can visualise this as the task of finding a mapping (or 'assignment') between the N samples and K clusters as pictured in Figure 9.1. In other words, we are aiming to find groups in the data (if they exist) such that we minimise the total distances of data points within clusters and maximise the distances between data points in different clusters, as pictured for $\mathbb{R}^3$ in Figure 9.1(b).

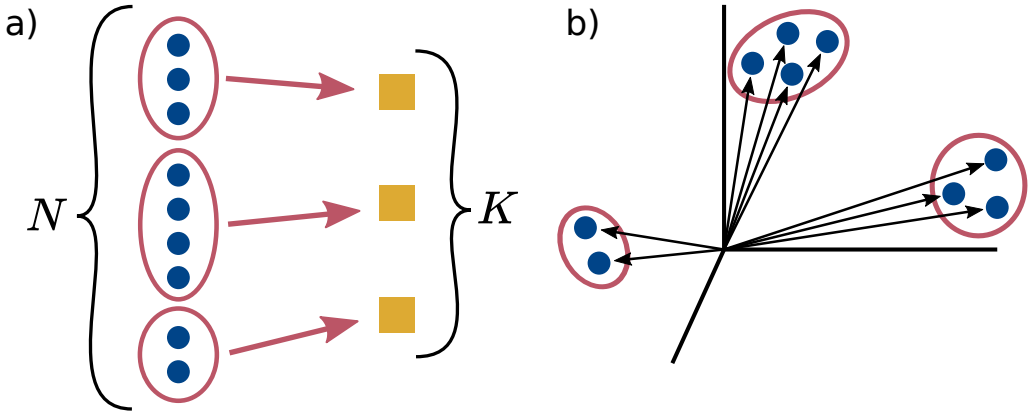


Figure 9.1. a) The clustering problem: Grouping N samples (blue dots) into K clusters (yellow boxes). b) If $\mathbf{x}^{(i)} \in \mathbb{R}^3$ (i.e. 3 variables), we can visualise the data as points in 3D space, which will be closer to each other within a cluster (red ellipses) as opposed to across clusters.

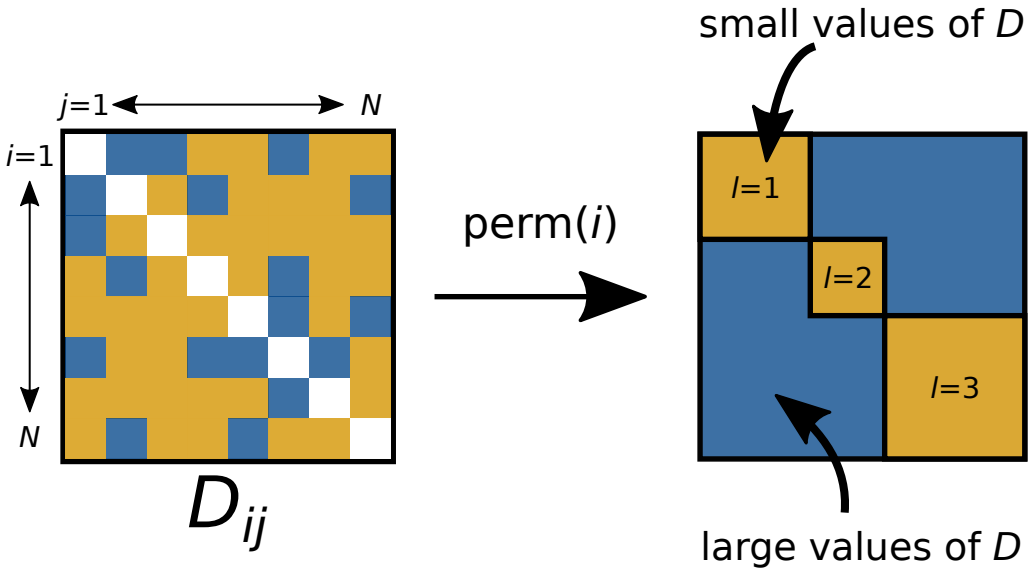


Figure 9.2. The clustering problem re-formulated as a matrix permutation problem. Starting from a distance matrix D, we find a permutation of the rows and columns, such that the resulting permuted matrix contains blocks of small values of D (in yellow) on the diagonal, whilst the off-diagonal sections contain mostly large values of D, shown in blue. Since we have $K = 3$ in this particular example, $l = 1, 2, 3$ represent the indices of each individual cluster. Note that the figure merely represents a cartoon-like depiction of the concept and should not be taken to be exact.

Another viewpoint on this problem is the following. Consider the $N \times N$ distance matrix D that contains all distances $D_{ij} := D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$. The clustering problem aims to find a reordering of the rows and columns of D according to some permutation $\text{perm}(i)$, such that the reordered matrix contains blocks of small values along the diagonal and large values off the block diagonals. **In essence, we want to produce a rearrangement of the matrix D that is maximally block-diagonal** - see Figure 9.2.

Using mathematical notation, given a specific clustering $\mathcal{C} = \{c_1, \ldots, c_K\}$ that matches each sample $\mathbf{x}^{(i)}$ to one of K clusters, we aim to **minimise the within-cluster distance**

W:

$$W(\mathcal{C}) = \frac{1}{2}\sum_{k=1}^{K}\sum_{i,j\in c_k} D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \tag{9.3}$$

or, equivalently, maximising the between-cluster distance $B(\mathcal{C})$:

$$B(\mathcal{C}) = \frac{1}{2}\sum_{k=1}^{K}\sum_{i\in c_k}\sum_{j\notin c_k} D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \tag{9.4}$$

This viewpoint shows that clustering can be seen as a *combinatorial optimisation* problem, i.e., the problem relies on finding some 'optimal' permutation of our samples (in the sense just defined) into groups among all possible permutations of this type. As we have already discussed (see Chapter 1), such problems can be 'hard', i.e., in order to find the true global optimum of the problem, one would have to check every single possible permutation, thus making the task of finding the best clustering an NP-hard problem, thus infeasible in practice even for datasets of moderate size (see *Hastie, Tibshirani, Friedman, The Elements of Statistical Learning*, Chap. 14, section 14.3.5 for a discussion). This implies that the most established clustering methods rely on some type of heuristics.

4. The K-means clustering method

In the basic clustering method named K-means, the goal is to find a *hard assignment* between the N samples and K clusters, i.e., an assigment whereby each sample belongs to one and only one cluster (unique membership).

After having decided on a specific value of K, the goal of K-means is to find the $N \times K$ *assignment matrix* taking the following form:

$$H_{N\times K} = \begin{bmatrix} 0 & 0 & 1 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & & 0 \\ 1 & 0 & 0 & \cdots & & 0 \\ & & \vdots & \ddots & & \\ 0 & \cdots & & & 0 & 1 \end{bmatrix}$$

Note that each row represents one sample data point and each column corresponds to one cluster. Each row contains exactly one entry equal to 1, assigning that particular data point to a particular cluster. Elsewhere, the row is simply filled with zeros.

Quick Aside. The assignment matrix H just defined summarises a hard assignment to a clustering $\mathcal{C} = \{c_l\}_{k=1}^{K}$ that is *exhaustive*. In other words, every single data point will be assigned to just one class, and orphan data points are *not* allowed. Mathematically, this can be described through the following properties (denoting each individual cluster with c_k for some $k = 1, \ldots, K$):

$$c_k \cap c_{k'} = \emptyset \qquad k \neq k'$$

$$\bigcup_{k=1}^{K} c_k = \{\mathbf{x}^{(i)}\}_{i=1}^{N}$$

In matrix-vector form, we have: $H_{ij} \in 0, 1$ with

$$H_{ij} \in 0, 1 \quad \text{with} \quad H\,\mathbf{1}_K = \mathbf{1}_N \quad \text{and} \quad \mathbf{1}_N^T H = (|c_1|, \ldots, |c_K|),$$

where the $\mathbf{1}$ are vectors of ones of the corresponding dimensions, e.g., $\mathbf{1}_N := \mathbf{1}_{N\times 1}$.

K-means algorithm. As explained above, we start with the data points $\mathbf{x}^{(i)} \in \mathbb{R}^p$ and a distance measure, in this case the squared Euclidean distance $D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2$. For a given clustering $\mathcal{C} = \{c_k\}_{k=1}^K$, we have an associated assignment matrix H. We then have the within distance:

$$W(\mathcal{C}) = \frac{1}{2} \sum_{k=1}^{K} \sum_{i,j \in c_k} \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2 \tag{9.5}$$

However, it is important to pay attention to unbalanced datasets, i.e., where the sizes of the subsets are expected to be quite different. Then one 'normalises' the above equation as follows (where $|c_k|$ is the number of data points in c_k):

$$W(\mathcal{C}) = \frac{1}{2} \sum_{k=1}^{K} \frac{1}{|c_k|} \sum_{i,j \in c_k} \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2 \tag{9.6}$$

We can re-formulate this in terms of the assignment matrix framework introduced above, so that the $W(\mathcal{C})$ in (9.6) can be rewritten in terms of H:

$$W(\mathcal{C}) = \frac{1}{2} \mathrm{Tr}\left[(H^T H)^{-1} \left[H^T_{K \times N} D_{N \times N} H_{N \times K} \right] \right] \tag{9.7}$$

(see derivation in the aside below).

Quick Aside: To understand better how (9.7) is obtained, we take each element of it and explicitly write out the corresponding matrices. The key insight is that the operation $H^T\, D\, H$ can be understood as effecting a sum over the blocks implicitly defined by H. Take for example $K = 3$, then we have:

$$H^T_{K \times N} D_{N \times N} H_{N \times K} = \begin{bmatrix} \clubsuit_1 & \square & \square \\ \square & \clubsuit_2 & \square \\ \square & \square & \clubsuit_3 \end{bmatrix},$$

where each $\clubsuit_k$ corresponds to the total 'within-cluster' distances for cluster c_k (which we are aiming to minimise):

$$\clubsuit_k = \sum_{i,j \in c_k} \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2$$

Furthermore, we have:

$$H^T H = \begin{bmatrix} \spadesuit_1 & 0 & 0 \\ 0 & \spadesuit_2 & 0 \\ 0 & 0 & \spadesuit_3 \end{bmatrix},$$

where each $\spadesuit_k$ corresponds exactly to the cardinality of each cluster c_k:

$$\spadesuit_k = |c_k|$$

Now we can see that (9.7) is indeed equivalent to (9.6).

The objective is to minimise $W(\mathcal{C})$, i.e., the diagonal elements of $(H^T H)^{-1}[H^T D H]$. Conversely, the off-diagonal elements can be written in similar matrix-vector notation. Consider the total sum of distances:

$$T = \frac{1}{2} \mathbf{1}_N^T D \mathbf{1}_N \tag{9.8}$$

$$= \frac{1}{2} \mathbf{1}_k^T [H^T D H] \mathbf{1}_K \tag{9.9}$$

where we have used $H_{N\times K}\mathbf{1}_{K\times 1} = \mathbf{1}_{N\times 1}$. Since T is fixed for the dataset, minimising $W(\mathcal{C})$ necessarily implies maximising $B = T - W(\mathcal{C})$, which in fact brings us back to the equation of the between-cluster distance (9.4).

K-means algorithm: The optimisation algorithm of K-means is as follows:

- **Step 0:** Given a number of clusters K, assign every sample to one of the K clusters at random.
- **Step 1:** Compute the *centroid* of each of the K clusters:
$$\mathbf{m}_k = \frac{1}{|c_k|}\sum_{i\in c_k}\mathbf{x}^{(i)}, \quad k = 1,\ldots,K$$
- **Step 2:** Reassign each $\mathbf{x}^{(i)}$ to the closest centroid. Mathematically speaking, we consider the cluster assigned to $\mathbf{x}^{(i)}$ at iteration t, which is denoted by $k_i^{(t)}$, and evaluate the following:
$$k_i^{(t+1)} = \underset{k}{\operatorname{argmin}}\|\mathbf{x}^{(i)} - \mathbf{m}_k\|^2$$
- **Iterate** steps 1 and 2 until convergence, i.e. $W(\mathcal{C})$ does not improve much (see below) or the assignments do not change.

In the above, one has introduced cluster centroids and formulated the algorithm in terms of them. Indeed, it can be shown that minimising the objective function (9.6) is equivalent to assigning data points to clusters by minimising their distance to the centroids of the cluster. Given a clustering $\mathcal{C} = \{c_k\}_{k=1}^K$, the objective function (9.6) can be rewritten as:

$$\begin{aligned}
W(\mathcal{C}) &= \frac{1}{2}\sum_{k=1}^{K}\frac{1}{|c_k|}\sum_{i,j\in c_k}\|(\mathbf{x}^{(i)} - \mathbf{m}_k) - (\mathbf{x}^{(j)} - \mathbf{m}_k)\|^2 \\
&= \sum_{k=1}^{K}\frac{1}{2}\frac{1}{|c_k|}\sum_{i,j\in c_k}\left[\|\mathbf{x}^{(i)} - \mathbf{m}_k\|^2 + \|\mathbf{x}^{(j)} - \mathbf{m}_k\|^2 - 2(\mathbf{x}^{(i)} - \mathbf{m}_k)(\mathbf{x}^{(j)} - \mathbf{m}_k)\right] \\
&= \sum_{k=1}^{K}\sum_{i\in c_k}\left[\|\mathbf{x}^{(i)} - \mathbf{m}_k\|^2 - \underbrace{(\mathbf{x}^{(i)} - \mathbf{m}_k)\frac{1}{|c_k|}\sum_{j\in c_k}(\mathbf{x}^{(i)} - \mathbf{m}_k)}_{=0}\right]
\end{aligned}$$

Leading to:

$$W(\mathcal{C}) = \frac{1}{2}\sum_{k=1}^{K}\frac{1}{|c_k|}\sum_{i,j\in c_k}\|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2 = \sum_{k=1}\sum_{i\in c_k}\|\mathbf{x}^{(i)} - \mathbf{m}_k\|^2$$

Note that this optimisation can be likened to a gradient method, since at every step $W(\mathcal{C})$ is decreased. However, since the problem is combinatorial, there is no guarantee that this procedure will converge to the global minimum (which would have to be found through complete enumeration). However, **the algorithm will converge to a local minimum, depending on the random initialisation** (the random assignment of points to clusters in Step 0 above).

Recognising the inherent lack of guarantees for global optimality in the algorithm, the standard procedure for K-means involves running K-means many times with different random initialisations to obtain an ensemble of optimised clusterings. One can then examine the ensemble of solutions found and evaluate the robustness of the obtained clusterings as indication that a partition into K groups is indeed capturing an intrinsic property of the

dataset. In many cases, one will then **choose the optimal clustering (according to the cost function W) from this ensemble of optimised clusterings** obtained from the runs of K-means.

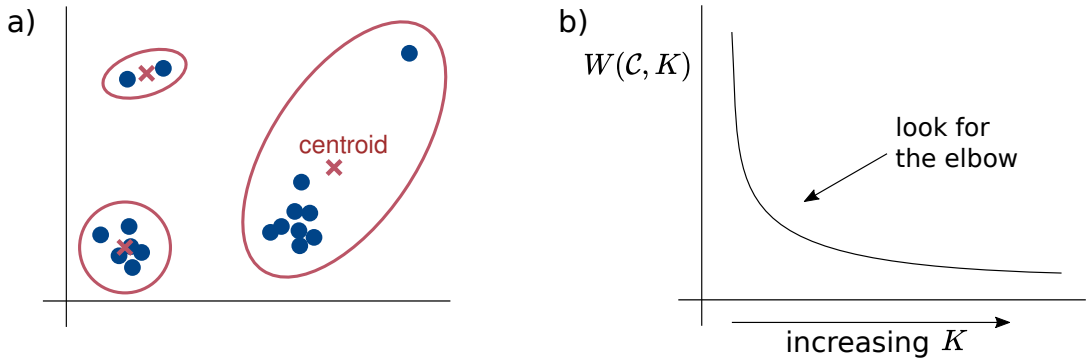


Figure 9.3. a) Cartoon depiction of the possible effect of outliers on K-means clustering: due to an outlier being included in the right cluster, the centroid of this cluster is not representative of the data. b) Optimisation of K: as K is increased, $W(\mathcal{C}, K)$ will decrease. The objective is to look for the 'elbow' of this curve, where increasing K brings no significant further improvement.

A couple of additional comments on K-means:

- **Outliers can pose problems for the K-means algorithm** as can be seen from Figure 9.3a. In some cases, it might happen that outliers will be counted towards a certain cluster, despite the large distance. In order to counteract this sensitivity to outliers, one can utilise an extension to the algorithm called K-medoids. In this method, instead of computing the centroid (which is not a sample point), a specific, representative sample point of those assigned to each cluster is used as the 'centroid' of the cluster, see for more details *Hastie, Tibshirani, Friedman, The Elements of Statistical Learning*, Chap. 14, section 14.3.10.
- Crucially, the question of choosing K remains: this is a hard and at the same time crucial task. This falls under the category of tuning the 'hyperparameters' of the model. In short, the answer to this question is to **try a range of increasing K's, until there is no longer any significant improvement in the clustering** (in terms of $W(\mathcal{C}, K)$). This is sometimes termed **'looking for the elbow'**. The basic idea is to use the principle of parsimony: if we view the clusters (e.g., the K centroids) as providing a concise description of the dataset, we want to find the smallest k that produces a small W. An illustration of this can be seen in Figure 9.3b, illustrated with a plateauing of $W(\mathcal{C}, K)$ as K is increased to larger values. Note that other measures of 'goodness' can also be used to decide on how to choose K, including measures of change in the assignment matrix H as K is increased. For a more detailed discussion, see also *Hastie, Tibshirani, Friedman, The Elements of Statistical Learning*, Chap. 14, section 14.3.11.

5. Hierarchical clustering

Hierarchical clustering is used broadly for data exploration, particularly in fields such as bioinformatics, as it produces an effective visualisation of the internal substructures of the data across different levels of resolution. In this sense, it gives a global description of the intrinsic organisation of the dataset without having to choose *a priori* a number of clusters.

We start with the usual description of our dataset: $\{\mathbf{x}^{(i)}\}_{i=1}^{N}$, and we also have some distance/similarity measure as defined above which produces our $N \times N$ matrix of distances between all points : $D_{ij} = D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$, $i, j = 1, \dots N$.

Key Message: Hierarchical clustering represents whole dataset $\{\mathbf{x}^{(i)}\}_{i=1}^{N}$ as a binary tree - a *dendrogram* - tracking how samples (leaf nodes) get agglomerated into larger groupings as one goes up to the root node (the whole dataset).

Each bifurcation (binary split) of the tree: *i)* corresponds to a merge of two groups of samples; *ii)* happens at a specific depth, enforcing a*strict hierarchy* (i.e., no cross-over of data points into other clusters as depth progresses).

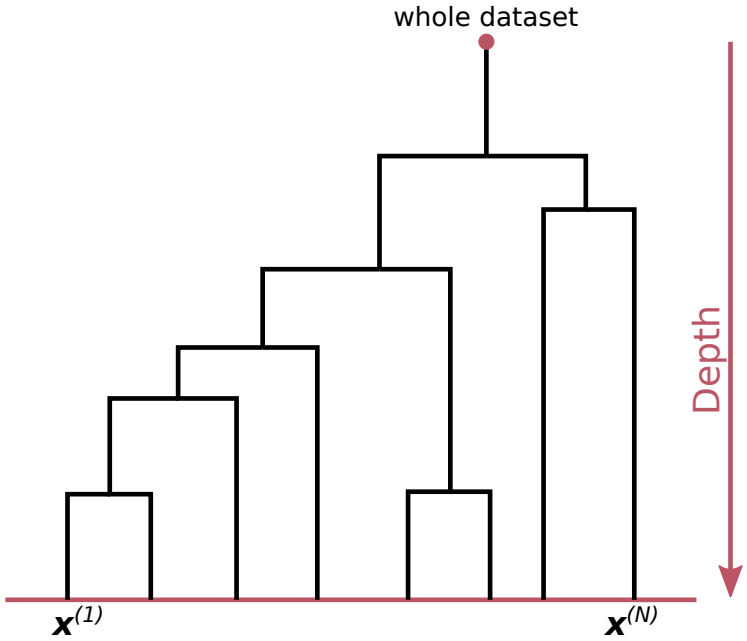


Figure 9.4. A typical dendogram, i.e., the binary tree summarising the groupings of the data samples obtained in hierarchical clustering as a function of the depth. The N leaves at the bottom are samples and the root at the top is the whole dataset. Starting at the top and moving down the tree, splits are made at various stages until finally, at the maximum depth, all individual data points are represented by a leaf of the tree each. Conversely, starting from the N leaves at the bottom (i.e., the N data points), we merge them into groups of increasing coarseness as go up towards the root.

A basic sketch of this idea can be seen in Figure 9.4. In order to produce the dendogram, there are in essence two approaches: bottom-up or top-down, i.e., agglomerative or divisive, respectively.

5.1. Agglomerative schemes for hierarchical clustering. Starting from each sample in its own individual cluster, the algorithm proceeds by merging samples into clusters of successive coarseness, to finally arrive at the whole dataset. So the aim is to reduce the number of clusters one-by-one from N down to 1. There are different versions of this agglomerative scheme depending on the criterion that decides the merge. Some of simplest and most popular of these criteria are:

Simple linkage (SL):. Given two clusters G, H, we consider:

$$d_{\mathrm{SL}}(G, H) = \min_{i \in G,\, j \in H} D_{ij}$$

This scheme can also be described as looking for the nearest neighbours across clusters. This minimum distance is then computed for every pair of clusters at a given depth level of the dendrogram and the minimal of such distances determines the next merge that will reduce the number of clusters by one.

Complete linkage (CL):. Given two clusters G, H, we consider:

$$d_{\mathrm{CL}}(G,H) = \max_{i\in G,\, j\in H} D_{ij}$$

This is essentially looking for furthest points across the two clusters G, H as a measure of distance between G and H, and then one looks for the minimal distance across all pairs of clusters present at a level of the dendrogram.

Average Linkage (GA):. The optimisation function in this case is:

$$d_{\mathrm{GA}}(G,H) = \frac{1}{N_G N_H} \sum_{i\in G, j\in H} D_{ij}$$

where N_G and N_H are the numbers of points respectively in cluster G and H. This computes the distance between clusters as an average over all pairs of points, and uses the distance between all pairs of clusters to decide on the next merge.

Ward's criterion: In its simplest form, Ward's criterion minimises the total within-cluster *variance*, i.e, at each level, we merge the pair of clusters such that the increase in total within-cluster variance is minimal.

A couple of comments about agglomerative schemes:

- The strictly hierarchical nature of the clusterings produced makes agglomerative methods **prone to variability due to sampling**, i.e., small amounts of noise in the data can lead to very different dendrograms.
- If just one partition is desired, it is difficult in most cases to decide what is the right level of resolution to pick in the dendrogram. The y-axis of the dendrogram is a function of the cluster distance used to decide on the merges (or splits), i.e., the amount by which the distance function has increased in the merge. Usually, stable levels of the dendrogram are picked, signalled by large jumps in the depth of the dendrogram between levels.
- As depth increases (i.e., as we move up the hierarchy, merging clusters), the total dissimilarity between grouped samples increases, the dissimilarity here being meant as the quantity minimised at each merge. This results in a *monotonic* increase in this dissimilarity moving from the dendrogram's leaves to the root.

5.2. Divisive schemes for hierarchical clustering. In opposition to the aforementioned agglomerative schemes, divisive schemes represent the 'top-down' approaches. This is usually done by recursive K-means with $K = 2$, which yields a binary tree, *but*: (1) monotonicity is not preserved; and (2) it depends on the initialisation at every split, leading to even more variable results.

6. Probabilistic clustering: Gaussian Mixtures

6.1. Probabilistic clustering. An alternative approach to clustering is that similarities in the data arise from some hidden common "cause". In this view, a cluster is then associated with a hidden variable, to form a so-called *latent variable model*. In brief, one

considers a model:

$$P(\boldsymbol{x}, z), \tag{9.10}$$

where $\boldsymbol{x}$ is an observed datapoint (with continuous features) and z is a categorical latent variable assuming K values corresponding to K clusters. According to this model, we can estimate the probability of a data point $\mathbf{x}^{(i)}$ belonging to the k-th cluster via

$$r_{ik} = P(z = k|\boldsymbol{x} = \mathbf{x}^{(i)}). \tag{9.11}$$

The above expression is called the **cluster probability or responsibility matrix**. For this reason, such methods are referred to as *probabilistic clustering* or *soft clustering*, in contrast to the hard clustering approaches we have seen in the previous sections. Any soft clustering can be converted to a hard one using the argmax discretisation we have seen for classification:

$$\underset{k}{\operatorname{argmax}}\, P(z = k|\boldsymbol{x} = \mathbf{x}^{(i)}). \tag{9.12}$$

An advantage of probabilistic clusterings is that they perform also density estimation and, as such, they yield a *generative model*: **a generative model is a model that can generate more data for each cluster by sampling** from the probability (9.11).

6.2. Gaussian mixture models. Mixture models are probabilistic models commonly used for clustering. The model consists of mixture components $P(\boldsymbol{x} = \mathbf{x}^{(i)}|z = k) = p_k(\mathbf{x}^{(i)}|\boldsymbol{\theta})$ and mixture weights $P(z = k) = \pi_k$ such that the marginal distribution of the data is a convex combination:

$$P(\boldsymbol{x} = \mathbf{x}^{(i)}) = \sum_{k=1}^{K} P(z = k)P(\boldsymbol{x} = \mathbf{x}^{(i)}|z = k) = \sum_{k=1}^{K} \pi_k p_k(\mathbf{x}^{(i)}|\boldsymbol{\theta}). \tag{9.13}$$

The mixture weights $\{\pi_k\}_{k=1}^K$ should satisfy:

$$0 \leq \pi_k \leq 1$$

$$\sum_{k=1}^{K} \pi_k = 1$$

in order to be valid probabilities. The model involves parameters $\boldsymbol{\theta}$ and mixture weights $\boldsymbol{\pi}$ that we need to choose given the data.

We have N observations $\{\mathbf{x}^{(i)}\}_{i=1}^N$, the cluster probabilities can then be computed as:

$$\begin{aligned} r_{ik}(\boldsymbol{\theta}) &= P(z = k|\boldsymbol{x} = \mathbf{x}^{(i)}, \boldsymbol{\theta}) \\ &= \frac{P(z = k, \boldsymbol{x} = \mathbf{x}^{(i)}, \boldsymbol{\theta})}{P(\boldsymbol{x} = \mathbf{x}^{(i)}, \boldsymbol{\theta})} = \frac{\pi_k p_k(\mathbf{x}^{(i)}|\boldsymbol{\theta})}{\sum_{k'=1}^{K} \pi_{k'} p_{k'}(\mathbf{x}^{(i)}|\boldsymbol{\theta})}. \end{aligned} \tag{9.14}$$

The most popular mixture models are Gaussian Mixture Models for which:

$$p_k(\boldsymbol{x}|\boldsymbol{\theta}) = (2\pi)^{-p/2} \det(\boldsymbol{\Sigma}_k)^{-1/2} \exp\left(-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_k)^T {\boldsymbol{\Sigma}_k}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_k)\right) \tag{9.15}$$

In unsupervised learning we do not have access to labels and therefore all parameters $\boldsymbol{\theta} = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1,..,K}$ should be chosen such that the model approximates the data as close as possible. A common choice is to **maximise the log-likelihood function of the probabilistic model**, in such a way that the probability modelled by the generative model closely matches the distribution of the data, as we discuss in the next section.

6.3. Expectation-Maximization algorithm (EM). In principle, the maximum likelihood estimate of the mixture is obtained, for numerical convenience, by maximising the log-likelihood:

(9.16)

$$\boldsymbol{\theta}_{\text{MLE}} = \underset{\boldsymbol{\theta}}{\text{argmax}} \log \mathcal{L}(\boldsymbol{\theta}) = \underset{\boldsymbol{\theta}}{\text{argmax}} \sum_{i=1}^{N} \log P(\boldsymbol{x} = \mathbf{x}^{(i)}|\boldsymbol{\theta}) = \underset{\boldsymbol{\theta}}{\text{argmax}} \sum_{i=1}^{N} \log \sum_{k=1}^{K} \pi_k p_k(\mathbf{x}^{(i)}|\boldsymbol{\theta}),$$

where $\mathcal{L}(\boldsymbol{\theta})$ is the data likelihood of the GMM, obtained from the data marginal distribution. In principle, the maximum likelihood estimate can be obtained using gradient ascent. The theoretical difficulty is that the likelihood involves a sum over the states of the latent variables z (standing from the K clusters). **EM is an iterative algorithm to find such a maximum likelihood solution by alternating 'expectation' and 'maximisation' steps**. It exploits the fact that if both $\boldsymbol{x}$ and z were fully observed, then the maximum likelihood estimate would be easy to compute.

Assume we had access to the full data $(\mathbf{x}^{(i)}, z^{(i)})$, $i = 1, ..., N$, then best parameters of the mixture model would be given by:

(9.17)

$$\pi_k(\boldsymbol{z}) = \frac{1}{N} \sum_{i=1}^{N} \delta_{z^{(i)},k}, \quad \boldsymbol{\mu}_k(\boldsymbol{z}) = \frac{1}{N_k} \sum_{i=1}^{N} \delta_{z^{(i)},k} \mathbf{x}^{(i)}, \quad \boldsymbol{\Sigma}_k(\boldsymbol{z}) = \frac{1}{N_k} \sum_{i=1}^{N} \delta_{z^{(i)},k} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)^T$$

where $\delta_{z^{(i)},k}$ is the indicator function $\mathbf{1}(z^{(i)} = k)$, N_k denotes the number of data points belonging to cluster k. However, in practice, the clustering $\boldsymbol{z}$ (i.e., the set assignments of a value k to the categorical variables $z^{(i)}$) is uncertain and we require another step to fill in this information. The general idea of the EM procedure is to replace the empirical averages with weighted averages whose weights correspond to the cluster probabilities.

The Expectation-Maximisation (EM) algorithm:

Initialise at random the parameters $\boldsymbol{\theta} = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1,..,K}$

At each iteration t, perform the E and M-steps, as follows:

Expectation step (E-step): uses current parameter values $\boldsymbol{\theta}^t = \{\pi_k^t, \boldsymbol{\mu}_k^t, \boldsymbol{\Sigma}_k^t\}_{k=1,..,K}$ to compute cluster probabilities $r_{ik}(\boldsymbol{\theta}^t)$ and a set of weights:

(9.18)
$$w_{ik}(\boldsymbol{\theta}^t) = \frac{r_{ik}(\boldsymbol{\theta}^t)}{\sum_{i'=1}^{N} r_{i'k}(\boldsymbol{\theta}^t)}$$

Maximisation step (M-step): updates the model's parameters for the next iteration $t+1$ maximising of the log-likelihood. One updates the mixture weight for the k-th component via the corresponding data-averaged cluster probability:

(9.19)
$$\pi_k^{t+1} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{E}_{z^{(i)}|\mathbf{x}^{(i)},\boldsymbol{\theta}^t}[\delta_{z^{(i)},k}] = \frac{1}{N} \sum_{i=1}^{N} r_{ik}(\boldsymbol{\theta}^t) = \frac{N_k}{N} \quad \text{Mixture weights}$$

One updates the parameters of the Gaussian mixture components using $w_{ik}(\boldsymbol{\theta}^t)$:

(9.20)
$$\boldsymbol{\mu}_k^{t+1} = \sum_{i=1}^{N} w_{ik}(\boldsymbol{\theta}^t) \mathbf{x}^{(i)} \quad \text{Gaussian means}$$

(9.21)
$$\boldsymbol{\Sigma}_k^{t+1} = \sum_{i=1}^{N} w_{ik}(\boldsymbol{\theta}^t)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k^{t+1})(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k^{t+1})^T \quad \text{Gaussian covariances}$$

which are simply a weighted version of (9.17).

Iterating between the E and M steps, we obtain a sequence of parameters $\{\boldsymbol{\theta}^t\}_{t=1,2,\ldots}$. The theoretical justification of this approach is that this sequence yields an increasing sequence marginal likelihoods $\mathcal{L}(\boldsymbol{\theta}^0) \leq \mathcal{L}(\boldsymbol{\theta}^1) \leq \ldots \leq \mathcal{L}(\boldsymbol{\theta}^t)$, guaranteeing **convergence to a local maximum** (see chapter 9 of Bishop, *Pattern Recognition And Machine Learning* for a proof). Since this optimum is only local, a common mitigation strategy is to start the algorithm multiple times from different initial guesses for the parameters $\boldsymbol{\theta}$, allowing one to get closer to the global optimum.

6.4. Relation to K-means. You may have wondered about the relation between the EM-algorithm for Gaussian mixture models and K-means, which are in fact related.

Assume to fix in a Gaussian mixture model $\boldsymbol{\Sigma} = \mathrm{diag}(1)$ and $\pi_k = 1/K$, so only the parameter to estimate is the cluster mean $\boldsymbol{\mu}_k$. Assume that we enforce a hard clustering taking the argmax of (9.14) via:

$$r_{ik}(\boldsymbol{\theta}) = \begin{cases} 1 & \text{if } k = \mathrm{argmax}_j \, r_{ij}(\boldsymbol{\theta}) = \mathrm{argmin}_j \, ||\mathbf{x}^{(i)} - \boldsymbol{\mu}_j||^2 \\ 0 & \text{otherwise} \end{cases} \tag{9.22}$$

This reduces the EM iteration of the mean (9.20) to the K-means algorithm, showing that Gaussian mixture models reduce to K-means in the limit of $\boldsymbol{\Sigma} = \mathrm{diag}(1)$, $\pi_k = 1/K$ and hard clustering assignments via the argmax discretisation. Conversely, **the EM-algorithm for Gaussian mixture models can be seen a generalisation of K-means to soft clustering**.

6.5. Choosing K. In K-means, as it is a *distance-based* method, we have seen that an optimal K can be set through the elbow rule, i.e., finding the elbow of the within-cluster distance as a function of the number of clusters K. In a GMM, as a distribution-based method, one appeals to criteria from information theory that include the likelihood of the distribution $\mathcal{L}(\boldsymbol{\theta})$, maximised during the training, as a measure of the goodness of fit. These criteria are the:

- Akaike Information Criterion (AIC):
  $AIC = 2N_{\boldsymbol{\theta}} - 2\ln(\mathcal{L}(\boldsymbol{\theta}))$
  where $N_{\boldsymbol{\theta}}$ denotes the number of parameters in the model.
- Bayesian Information Criterion (BIC):
  $BIC = N_{\boldsymbol{\theta}} lnN - 2\ln(\mathcal{L}(\boldsymbol{\theta}))$

where we have stressed that the log of the likelihood here is meant as the natural log. They are designed to balance the goodness of fit, measured by the log-likelihood maximised by the learnt parameters, and the model complexity (measured by the overall number of parameters in the model), in such a way as to discourage overfitting. We see that BIC replaces the factor 2 in AIC by $\ln N$: for typical sample sizes, let's say $N > e^2 \sim 7.4$, BIC will tends to penalise complex models more heavily than AIC; BIC adapts this model complexity penalty to the sample size, making it particularly severe for small datasets. You can read more about the use of AIC and BIC in model selection in Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*, Chap. 7.

The optimal K can be set by looking for the *minimum* of the AIC or BIC.

7. Comparing clusterings

A brief note on the comparison of clusterings is in order. In many cases, one needs to compare how similar two clusterings are. This topic is related to the confusion and

contingency matrices we have seen for classification (Chapter 3), but here, again, we do not have a ground truth against which we are comparing, and clusterings can have different numbers of groups.

Typically, the way to compare clusterings is through the creation of a **count matrix** that matches the number of coincident elements in each cluster of the two clusterings. Then a summary measure is compiled as a quality of the overlap of elements across clusters. Main examples are the following.

Adjusted Rand Index[1]**.** Given a set of N elements, and two groupings or partitions (e.g. clusterings) of these elements with respectively r and s elements, namely $X = \{X_1, X_2, \ldots, X_K\}$ and $Y = \{Y_1, Y_2, \ldots, Y_{K'}\}$, the overlap between X and Y can be summarized in a contingency table $[n_{ij}]$ where each entry n_{ij} denotes the number of objects in common between X_i and Y_j : $n_{ij} = |X_i \cap Y_j|$.

	Y_1	Y_2	$\cdots$	$Y_{K'}$	sums
X_1	n_{11}	n_{12}	$\cdots$	$n_{1K'}$	a_1
X_2	n_{21}	n_{22}	$\cdots$	$n_{2K'}$	a_2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
X_K	n_{K1}	n_{K2}	$\cdots$	$n_{KK'}$	a_K
sums	b_1	b_2	$\cdots$	$b_{K'}$	

The Adjusted Rand Index is then defined by:

$$ARI = \frac{\sum_{ij} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}\right] \Big/ \binom{N}{2}}{\frac{1}{2}\left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}\right] \Big/ \binom{N}{2}}$$

Mutual Information and Adjusted Mutual Information[2]**.** Given two clusterings X and Y of size K and K' respectively, the Mutual Information (MI) score is defined as:

$$MI(X, Y) = \sum_{i=1}^{K} \sum_{j=1}^{K'} P_{XY}(i, j) \log \frac{P_{XY}(i, j)}{P_X(i) P_Y(j)}$$

where $P_{XY}(i, j)$ denotes the probability that a point belongs to both the cluster X_i in X and cluster Y_j in Y:

$$P_{XY}(i, j) = \frac{|X_i \cap Y_i|}{N}$$

N being the total number of data points.

The Adjusted Mutual Information (AMI) is an adjustment of MI to account for chance. It accounts for the fact that the MI is generally higher for two clusterings with a larger number of clusters, regardless of whether there is actually more information shared.[3] It is defined as:

$$AMI(X, Y) = \frac{MI(X, Y) - E\{MI(X, Y)\}}{\max\{H(X), H(Y)\} - E\{MI(X, Y)\}}$$

[1] https://en.wikipedia.org/wiki/Rand_index#Adjusted_Rand_index

[2] https://en.wikipedia.org/wiki/Adjusted_mutual_information

[3] https://scikit-learn.org/stable/modules/generated/sklearn.metrics.adjusted_mutual_info_score.html

Here, we have the following definitions:

$$H(X) = -\sum_{i=1}^{K} P_X(i) \log P_X(i) \qquad \text{The clustering entropy;}$$

$$E\{MI(X,Y)\} = \sum_{i=1}^{K}\sum_{j=1}^{K'} \sum_{n_{ij}=(a_i+b_j-N)^+}^{\min(a_i,b_j)} \frac{n_{ij}}{N} \log\left(\frac{N \cdot n_{ij}}{a_i b_j}\right) \times \frac{a_i! b_j! (N-a_i)! (N-b_j)!}{N! n_{ij}! (a_i - n_{ij})! (b_j - n_{ij})! (N - a_i - b_j + n_{ij})!}$$

Normalised Variation of Information[4]**.** From the Mutual Information $MI(X,Y)$ and the Entropy measures $H(X)$ and $H(Y)$ we can directly compute the Normalised Variation of Information (NVI) given by:

$$NVI(X,Y) = \frac{H(X) + H(Y) - 2MI(X,Y)}{H(X) + H(Y) - MI(X,Y)} = \frac{VI(X,Y)}{H(X,Y)}$$

where $VI(X,Y) = H(X) + H(Y) - 2MI(X,Y)$ is the variation of information and $H(X,Y) = H(X) + H(Y) - MI(X,Y)$ is the joint entropy of the partitions.

The NVI is a metric on the space of partitions that is easier to interpret, as it is bounded between 0 and 1: $NVI = 0$ implies that the two clusterings are identical, $NVI = 1$ indicates that the clusterings are completely independent.

[4] http://arxiv.org/abs/q-bio/0311039